

LA CONQUISTA

Costes y justificación de estructuras de datos

Metodología de la Programación y Estructuras de Datos

Grado en Matemáticas y Computación - 1.º curso

Universidad de Alcalá - Grupo H08G05

Asier López Rodero, Mateo Ramos Gómez y Marcos Fernández Pérez

Mayo de 2026

1. Objetivo del documento

Este documento amplía la justificación de las estructuras de datos propias utilizadas en La Conquista. Su finalidad es mostrar que las estructuras no se han incorporado de forma decorativa, sino vinculadas a necesidades reales del juego: representación de habitaciones, mapa global, turnos, inventario, búsqueda de caminos, recorrido de entidades y validación de reglas.

Los costes se expresan en notación O grande de forma orientativa. Para una práctica de primer curso no se pretende demostrar formalmente cada algoritmo, sino razonar por qué cada estructura es adecuada para el tamaño y comportamiento del juego.

2. Restricciones asumidas

- No se utilizan ArrayList, HashMap, LinkedList ni estructuras estándar equivalentes para resolver las partes evaluadas de Estructuras de Datos.
- Las habitaciones se modelan como matrices propias y el mapa de habitaciones como grafo propio.
- Las colecciones del dominio se recorren mediante estructuras propias, priorizando claridad, trazabilidad y control del código.
- La eficiencia se evalúa de forma proporcional al tamaño del juego: pocas habitaciones, inventarios pequeños y enemigos limitados por sala.

3. Tabla general de estructuras

Estructura propia	Uso en el proyecto	Operaciones principales	Coste aproximado
ListaEnlazadaPropia	Inventario, enemigos, objetos, habitaciones, adyacencias y recorridos de colecciones.	insertar, eliminar, buscar, recorrer	insertar al final $O(n)$, eliminar/buscar $O(n)$, recorrer $O(n)$
PilaPropia	Soporte LIFO para recorridos auxiliares, reconstrucción/inversión de caminos y estructura lineal exigida.	push, pop, cima, estaVacia	$O(1)$ en push/pop/cima
ColaPropia	BFS en matriz/grafos y procesamiento ordenado de turnos o entidades.	encolar, desencolar, frente	$O(1)$ en encolar/desencolar si mantiene referencias a inicio/fin
ListaCircularPropia	Recorridos cíclicos y rotación de elementos cuando se procesan entidades o acciones repetibles.	avanzar, insertar, recorrer ciclo	avanzar $O(1)$, insertar $O(n)$ según posición, recorrer $O(n)$
MatrizPropia	Representación interna de cada habitación y acceso a celdas del mapa.	obtener celda, modificar celda, recorrer tablero	acceso $O(1)$ si hay acceso directo; recorrido $O(f \cdot c)$
GrafoPropio	Conexión de habitaciones y cálculo de ruta hacia la salida.	añadir vértice, añadir arista, vecinos, BFS	vecinos $O(\text{grado})$, BFS $O(V+E)$
ArbolPropio	Organización jerárquica de decisiones, acciones o categorías del juego.	insertar, buscar, recorrer	$O(h)$ búsqueda/inserción; $O(n)$ recorrido

4. Costes por operaciones reales del juego

Operación del juego	Estructuras implicadas	Coste	Justificación
Pintar la habitación actual	MatrizPropia + listas de entidades	$O(f \cdot c + e + o)$	La interfaz recorre celdas y consulta entidades/objetos para mostrar jugador, enemigos, puertas, trampas y salida.
Validar movimiento del jugador	MatrizPropia	$O(1)$	Se comprueba la celda destino: límites, muro, ocupación, puerta o trampa.
Calcular casillas alcanzables	MatrizPropia + ColaPropia	$O(f \cdot c)$	BFS sobre celdas transitables; cada celda se visita como máximo una vez.
Procesar enemigos de una habitación	ListaEnlazadaPropia/ ListaCircularPropia	$O(e)$	Se recorre cada enemigo vivo de la sala y se decide movimiento o ataque.
Gestionar inventario	ListaEnlazadaPropia	$O(n)$	Buscar, usar o equipar objetos requiere recorrer un inventario de tamaño pequeño.
Comprar/ver ruta a salida	GrafoPropio + ColaPropia	$O(V+E)$	BFS en el grafo de habitaciones hasta la sala con salida.
Cambiar de habitación	GrafoPropio + MatrizPropia	$O(\text{grado} + 1)$	Se comprueba conexión válida y se coloca al jugador en la celda de entrada.
Guardar partida	Listas + matriz + entidades	$O(\text{total de entidades y celdas relevantes})$	Se serializa el estado necesario: jugador, inventario, enemigos, objetos, habitación y turnos.
Cargar partida	JSON + estructuras propias	$O(\text{total de entidades cargadas})$	Se reconstruye el estado y se vuelven a poblar colecciones y habitaciones.
Ejecutar pruebas de estructuras	Todas las estructuras	Variable	Cada test comprueba operaciones básicas y casos límite de la estructura correspondiente.

5. Justificación individual

5.1 Lista enlazada propia

La lista enlazada es adecuada para colecciones de tamaño variable: inventario, enemigos de una habitación, objetos, habitaciones cargadas y listas de adyacencia del grafo. Aunque el acceso por índice es $O(n)$, en este proyecto se prioriza recorrer secuencialmente y el tamaño de las colecciones es reducido. Su ventaja principal es sustituir estructuras estándar prohibidas manteniendo una implementación comprensible y evaluable.

5.2 Pila propia

La pila representa el patrón LIFO. En el diseño del proyecto resulta útil para recorridos auxiliares, reconstrucción inversa de caminos o históricos simples de acciones. Aunque no sea la estructura más visible en la interfaz, demuestra dominio de una estructura lineal diferente a la lista y permite justificar operaciones de coste constante.

5.3 Cola propia

La cola es una de las estructuras más importantes porque se ajusta naturalmente al procesamiento FIFO. Se utiliza para BFS en matriz y grafo, y también resulta coherente para modelar turnos o entidades que actúan en orden. Su coste $O(1)$ al encolar y desencolar hace que el algoritmo sea eficiente y fácil de explicar.

5.4 Lista circular propia

La lista circular permite recorrer elementos de forma cíclica sin volver manualmente al inicio. Encaja con mecánicas repetitivas como rotación de enemigos, turnos o secuencias de acciones. Es útil para mostrar que el grupo distingue entre una lista lineal y una estructura preparada para ciclos.

5.5 Matriz propia

La matriz es esencial porque cada habitación no es solo un nodo del grafo, sino un tablero interno. En ella se comprueban posiciones, muros, puertas, trampas, objetos, enemigos y salida. El coste de recorrer toda la habitación $O(f \cdot c)$ es aceptable porque los tableros son pequeños y permite una representación muy clara en JavaFX.

5.6 Grafo propio

El grafo representa el mapa global de habitaciones. Cada habitación es un vértice y cada conexión o puerta es una arista. Esta elección permite calcular rutas entre salas, justificar el camino mínimo y separar el movimiento dentro de una habitación del movimiento entre habitaciones. El coste BFS $O(V+E)$ es adecuado para una campaña de tamaño limitado.

5.7 Árbol propio

El árbol aporta una estructura jerárquica para organizar decisiones, acciones o categorías del juego. Es especialmente defendible desde el punto de vista metodológico porque permite representar opciones principales y subopciones de forma ordenada. Su coste depende de la altura del árbol, por lo que un árbol equilibrado mejora las búsquedas frente a una estructura degenerada.

6. Relación entre coste y diseño

La selección de estructuras no busca optimizar prematuramente un juego pequeño, sino demostrar una elección razonada. En un proyecto de mayor tamaño, algunas operaciones podrían necesitar índices o mapas para acelerar búsquedas; sin embargo, en esta práctica esas estructuras estándar están restringidas y el tamaño de los datos hace viable el uso de listas y recorridos lineales.

El punto más relevante es la separación de niveles: matriz para el interior de cada habitación y grafo para la red de habitaciones. Esta separación evita confundir moverse por celdas con cambiar de sala y permite explicar dos algoritmos diferentes: BFS en celdas y BFS en habitaciones.

7. Conclusión

Las estructuras propias cubren las categorías exigidas y se relacionan con operaciones concretas del juego. La solución no se limita a implementar clases aisladas: las estructuras sostienen el movimiento, los turnos, la persistencia, la interfaz, la ruta mínima y la organización interna del dominio. Por ello, el diseño es adecuado para la evaluación de Estructuras de Datos y coherente con la memoria del proyecto.